

Introducing Google AI Edge Portal (<https://ai.google.dev/edge/ai-edge-portal>): Benchmark Edge AI at scale. Sign-up (<https://docs.google.com/forms/d/e/1FAIpQLSfTcGPycQve8TLAsfH46pBIXBZe9FrgJAClwbF7DeL1LgVn4Q/viewform>) to request access during private preview.

NPU acceleration with LiteRT

LiteRT provides a unified interface to use Neural Processing Units (NPUs) without requesting you to navigate vendor-specific compilers, runtimes, or library dependencies. Using LiteRT for NPU acceleration boosts performance for real-time and large-model inference and minimizes memory copies through zero-copy hardware buffer usage.

Get Started

Classical ML models GenAI models (#genai-models)
(#classical-ml-models)

For classical ML models, see the following demo applications.

- Image segmentation Kotlin App
(https://github.com/google-ai-edge/litert-samples/tree/main/compiled_model_api/image_segmentation/kotlin_npu)
: AOT
(https://github.com/google-ai-edge/litert-samples/tree/main/compiled_model_api/image_segmentation/kotlin_npu/android)
and on-device(JIT)
(https://github.com/google-ai-edge/litert-samples/tree/main/compiled_model_api/image_segmentation/kotlin_npu/android_jit)
compilation.
- Image segmentation C++ App
(https://github.com/google-ai-edge/litert-samples/tree/main/compiled_model_api/image_segmentation/c++_segmentation)
: AOT and on-device (JIT) compilation in the same app.

NPU Vendors

LiteRT supports NPU acceleration with the following vendors:

[Google Tensor](#)[Qualcomm AI Engine Direct...](#)[MediaTek NeuroPilot...](#)[More ▾](#)

#google-tensor

- Support AOT execution through the `CompiledModel` API.
- See [Google Tensor](https://developers.google.com/edge/litert/next/tensor-sdk) (<https://developers.google.com/edge/litert/next/tensor-sdk>) for setup details.

 **Beta:** Google Tensor SDK doesn't yet support on-device (JIT) compilation.

AOT and on-device compilation

LiteRT NPU supports both AOT and on-device compilation to meet your specific deployment requirements:

- **Offline (AOT) compilation:** This is best suited for large, complex models where the target SoC is known. Compiling ahead-of-time significantly reduces initialization costs and lowers memory usage when the user launches your app.
- **Online (on-device) compilation:** Also known as JIT compilation. This is ideal for platform-agnostic model distribution of small models. The model is compiled on the user's device during initialization, requiring no extra preparation step but incurring a higher first-run cost.

Here's how you can deploy your model using both AOT or on-device compilation options:

Step 1: AOT Compilation for the target NPU SoCs

You can use the LiteRT AOT (ahead of time) Compiler to compile your .tflite model to the supported SoCs. You can also target multiple SoC vendors and versions simultaneously within a single compilation process. See more details in this [LiteRT AOT Compilation notebook](#)

(https://github.com/google-ai-edge/litert-samples/blob/main/compiled_model_api/colab/LiteRT_AOT_Compilation_Tutorial.ipynb)

. While optional, AOT compilation is highly recommended for larger models to reduce on-device initialization time. This step is not required for on-device compilation.

Step 2: Deploy with Google Play if on Android

On Android, use Google [Play for On-device AI \(PODAI\)](https://developer.android.com/google/play/on-device-ai).

(<https://developer.android.com/google/play/on-device-ai>) to deploy the model and NPU runtime libraries with your app.

- **For models of on-device compilation:** add the original .tflite model file directly into your app's assets/ directory.
 - See example implementation in the [Segmentation on-device compilation Kotlin App](https://github.com/google-ai-edge/litert-samples/tree/main/compiled_model_api/image_segmentation/kotlin_npu/android_jit) (https://github.com/google-ai-edge/litert-samples/tree/main/compiled_model_api/image_segmentation/kotlin_npu/android_jit)
- **For models of AOT compilation:** use LiteRT to export your compiled models into a single Google [Play AI Pack](https://developer.android.com/google/play/on-device-ai#get_started_with_ai_packs) (https://developer.android.com/google/play/on-device-ai#get_started_with_ai_packs). You then upload the AI pack to Google Play to automatically deliver the correct compiled models to users' devices.
 - See [LiteRT AOT Compilation notebook](https://github.com/google-ai-edge/litert-samples/blob/main/compiled_model_api/colab/LiteRT_AOT_Compilation_Tutorial.ipynb) (https://github.com/google-ai-edge/litert-samples/blob/main/compiled_model_api/colab/LiteRT_AOT_Compilation_Tutorial.ipynb) for instructions on exporting compiled models into a Play AI Pack.
 - See example implementation in the [Segmentation AOT compilation Kotlin App](https://github.com/google-ai-edge/litert-samples/tree/main/compiled_model_api/image_segmentation/kotlin_npu/android) (https://github.com/google-ai-edge/litert-samples/tree/main/compiled_model_api/image_segmentation/kotlin_npu/android)
- **For NPU runtime libraries,** use [Play Feature Delivery](https://developer.android.com/guide/playcore/feature-delivery) (<https://developer.android.com/guide/playcore/feature-delivery>) to distribute the correct runtime libraries to users' devices.

See the following sections about how to deploy with [Play AI Pack](https://developer.android.com/google/play/on-device-ai#get_started_with_ai_packs)

(https://developer.android.com/google/play/on-device-ai#get_started_with_ai_packs) and [Play Feature Delivery](https://developer.android.com/guide/playcore/feature-delivery) (<https://developer.android.com/guide/playcore/feature-delivery>).

Deploy AOT models with Play AI Pack

The following steps guide you through deploying your AOT compiled models using Play AI Packs.

Add AI Pack to the project

Import AI Packs into the Gradle project by copying the AI Pack(s) to the root directory of the Gradle project. For example:

```
my_app/  
    ...  
    ai_packs/  
        my_model/...  
        my_model_mtk/...
```

Add each AI Pack to the Gradle build config:

```
// my_app/ai_packs/my_model/build.gradle.kts  
  
plugins { id("com.android.ai-pack") }  
  
aiPack {  
    packName = "my_model" // ai pack dir name  
    dynamicDelivery { deliveryType = "on-demand" }  
}  
  
// Add another build.gradle.kts for my_model_mtk/ as well
```

Add AI Packs to the Gradle config

Copy `device_targeting_configuration.xml` from the generated AI Packs to the directory of the main app module. Then update `settings.gradle.kts`:

```
// my_app/setting.gradle.kts  
  
...  
// AI Packs  
include(":ai_packs:my_model")  
include(":ai_packs:my_model_mtk")
```

Update `build.gradle.kts`:

```
// my_app/build.gradle.kts

android {
    ...

    defaultConfig {
        ...
        // API level 31+ is required for NPU support.
        minSdk = 31
    }

    // AI Packs
    assetPacks.add(":ai_packs:my_model")
    assetPacks.add(":ai_packs:my_model_mtk")
}
```

Configure AI Pack for on-demand delivery

On-demand delivery lets you request the model at runtime, which is useful if the model is only required for certain user-flows. Your model will be downloaded to your app's internal storage space. With the Android AI Pack feature configured in the `build.gradle.kts` file, check the device capabilities. See also [instructions](https://developer.android.com/guide/playcore/feature-delivery) (<https://developer.android.com/guide/playcore/feature-delivery>) for **install-time delivery** and **fast-follow delivery** from PODAI.

```
val env = Environment.create(BuiltinNpuAcceleratorProvider(context))

val cpuGpuModelProvider =
    ModelProvider.staticModel(
        ModelProvider.Type.ASSET,
        "model/my_model_cpu_gpu.tflite",
        if (accelerator != Accelerator.NPU) accelerator else Accelerator.CPU,
    )

val qualcommNpuModelProvider =
    AiPackModelProvider(context, "my_model", "model/my_model.tflite")
{
    buildSet {
        if (
            accelerator == Accelerator.NPU && NpuCompatibilityChecker.Qualcomm.isDeviceSu
        )
            add(Accelerator.NPU)
    }
}
```

```

    }

    val mtkNpuModelProvider =
        AiPackModelProvider(context, "my_model_mtk", "model/my_model.tflite")
    {
        buildSet {
            if (
                accelerator == Accelerator.NPU && NpuCompatibilityChecker.Mediatek.isDeviceSu
            )
                add(Accelerator.NPU)
        }
    }

    val googleTensorTpuModelProvider =
        AiPackModelProvider(context, "my_model", "model/my_model.tflite")
    {
        buildSet {
            if (accelerator == Accelerator.NPU &&
                NpuCompatibilityChecker.GoogleTensor.isDeviceSupported()
            )
                add(Accelerator.NPU)
        }
    }

    val samsungNpuModelProvider =
        AiPackModelProvider(context, "my_model", "model/my_model.tflite")
    {
        buildSet {
            if (
                accelerator == Accelerator.NPU && NpuCompatibilityChecker.Samsung.isDeviceSup
            )
                add(Accelerator.NPU)
        }
    }

    val aiPackModelProvider =
        ModelSelector(cpuGpuModelProvider, mtkNpuModelProvider, qualcommNpuModelProvi
            .selectModel(env)

    val compiledModel = CompiledModel.create(
        model.getPath(),
        CompiledModel.Options(model.getCompatibleAccelerators()),
        env,
    )

```

Deploy NPU runtime libraries with Play Feature Delivery

Play Feature Delivery (<https://developer.android.com/guide/playcore/feature-delivery>) supports multiple delivery options to optimize the initial download size, including install-time delivery, on-demand delivery, conditional delivery, and instant delivery. Here, we show the basic install-time delivery guide.

Add NPU runtime libraries to the project

Download `litert_npu_runtime_libraries.zip` from the latest release

(<https://github.com/google-ai-edge/LiteRT/releases/>) for AOT compilation or

`litert_npu_runtime_libraries_jit.zip` from the latest release

(<https://github.com/google-ai-edge/LiteRT/releases/>) for on-device compilation, and unpack it in the root directory of the project:

```
my_app/  
...  
litert_npu_runtime_libraries/  
  google_tensor_runtime/...  
  samsung_runtime/...  
  mediatek_runtime/...  
  qualcomm_runtime_v69/...  
  qualcomm_runtime_v73/...  
  qualcomm_runtime_v75/...  
  qualcomm_runtime_v79/...  
  qualcomm_runtime_v81/...  
  fetch_qualcomm_library.sh
```

Run the script to download the NPU support libraries. For example, run the following for Qualcomm NPUs:

```
$ ./litert_npu_runtime_libraries/fetch_qualcomm_library.sh
```

Add NPU runtime libraries to the Gradle config

Copy `device_targeting_configuration.xml` from the generated AI Packs to the directory of the main app module. Then update `settings.gradle.kts`:

```
// my_app/setting.gradle.kts
```

```
...
// NPU runtime libraries
include(":litert_npu_runtime_libraries:runtime_strings")
include(":litert_npu_runtime_libraries:google_tensor_runtime")
include(":litert_npu_runtime_libraries:samsung_runtime")
include(":litert_npu_runtime_libraries:mediatek_runtime")
include(":litert_npu_runtime_libraries:qualcomm_runtime_v69")
include(":litert_npu_runtime_libraries:qualcomm_runtime_v73")
include(":litert_npu_runtime_libraries:qualcomm_runtime_v75")
include(":litert_npu_runtime_libraries:qualcomm_runtime_v79")
include(":litert_npu_runtime_libraries:qualcomm_runtime_v81")
```

Update **build.gradle.kts**:

```
// my_app/build.gradle.kts
```

```
android {
    ...
    defaultConfig {
        ...
        // API level 31+ is required for NPU support.
        minSdk = 31

        // NPU only supports arm64-v8a
        ndk { abiFilters.add("arm64-v8a") }
        // Needed for Qualcomm NPU runtime libraries
        packaging { jniLibs { useLegacyPackaging = true } }
    }

    // Device targeting
    bundle {
        deviceTargetingConfig = file("device_targeting_configuration.xml")
        deviceGroup {
            enableSplit = true // split bundle by #group
            defaultGroup = "other" // group used for standalone APKs
        }
    }

    // NPU runtime libraries
    dynamicFeatures.add(":litert_npu_runtime_libraries:google_tensor_runtime")

    dynamicFeatures.add(":litert_npu_runtime_libraries:samsung_runtime")
}
```



```

dynamicFeatures.add(":litert_npu_runtime_libraries:mediatek_runtime")
dynamicFeatures.add(":litert_npu_runtime_libraries:qualcomm_runtime_v69")
dynamicFeatures.add(":litert_npu_runtime_libraries:qualcomm_runtime_v73")
dynamicFeatures.add(":litert_npu_runtime_libraries:qualcomm_runtime_v75")
dynamicFeatures.add(":litert_npu_runtime_libraries:qualcomm_runtime_v79")
dynamicFeatures.add(":litert_npu_runtime_libraries:qualcomm_runtime_v81")
}

dependencies {
    // Dependencies for strings used in the runtime library modules.
    implementation(project(":litert_npu_runtime_libraries:runtime_strings"))
    ...
}

```

Step 3: Inference on NPU using LiteRT Runtime

LiteRT abstracts away the complexity of developing against specific SoC versions, letting you run your model on the NPU with just a few lines of code. It also provides a robust, built-in fallback mechanism: you can specify CPU, GPU, or both as options, and LiteRT will automatically use them if the NPU is unavailable. Conveniently, AOT compilation also supports fallback. It provides partial delegation on NPU where unsupported subgraphs seamlessly run on CPU or GPU as specified.

Run in Kotlin

See example implementation in the following demo apps:

- [Segmentation on-device compilation Kotlin App](https://github.com/google-ai-edge/litert-samples/tree/main/compiled_model_api/image_segmentation/kotlin_npu/android_jit)
(https://github.com/google-ai-edge/litert-samples/tree/main/compiled_model_api/image_segmentation/kotlin_npu/android_jit)
- [Segmentation AOT compilation Kotlin App](https://github.com/google-ai-edge/litert-samples/tree/main/compiled_model_api/image_segmentation/kotlin_npu/android)
(https://github.com/google-ai-edge/litert-samples/tree/main/compiled_model_api/image_segmentation/kotlin_npu/android)

Add Android dependencies

You can add the latest LiteRT Maven package to your build.gradle dependencies:

```

dependencies {
    ...
}

```

```
implementation("com.google.ai.edge.litert:litert:+")
}
```

Runtime integration

```
// 1. Load model and initialize runtime.
// If NPU is unavailable, inference will fallback to GPU.
val model =
    CompiledModel.create(
        context.assets,
        "model/mymodel.tflite",
        CompiledModel.Options(Accelerator.NPU, Accelerator.GPU)
    )

// 2. Pre-allocate input/output buffers
val inputBuffers = model.createInputBuffers()
val outputBuffers = model.createOutputBuffers()

// 3. Fill the first input
inputBuffers[0].writeFloat(...)

// 4. Invoke
model.run(inputBuffers, outputBuffers)

// 5. Read the output
val outputFloatArray = outputBuffers[0].readFloat()
```

Run in C++ cross-platform

See example implementation in the [Asynchronous segmentation C++ App](https://github.com/google-ai-edge/litert-samples/tree/main/compiled_model_api/image_segmentation/c++_segmentation)
(https://github.com/google-ai-edge/litert-samples/tree/main/compiled_model_api/image_segmentation/c++_segmentation)

.

Bazel Build dependencies

C++ users must build the dependencies of the application with LiteRT NPU acceleration. The `cc_binary` rule that packages the core application logic (e.g., `main.cc`) requires the following runtime components:

- **LiteRT C API shared library:** the data attribute must include the LiteRT C API shared library (`//litedrt/c:litedrt_runtime_c_api_shared_lib`) and the vendor-specific dispatch shared object for the NPU (`//litedrt/vendors/qualcomm/dispatch:dispatch_api_so`).
- **NPU-specific backend libraries:** For example, the Qualcomm AI RT (QAIRT) libraries for the Android host (like `libQnnHtp.so`, `libQnnHtpPrepare.so`) and the corresponding Hexagon DSP library (`libQnnHtpV79Skel.so`). This ensures that the LiteRT runtime can offload computations to the NPU.
- **Attribute dependencies:** the `deps` attribute links against essential compile-time dependencies, such as LiteRT's tensor buffer (`//litedrt/cc:litedrt_tensor_buffer`) and the API for the NPU dispatch layer (`//litedrt/vendors/qualcomm/dispatch:dispatch_api`). This enables your application code to interact with the NPU through LiteRT.
- **Model files and other assets:** Included through the data attribute.

This setup allows your compiled binary to dynamically load and use the NPU for accelerated machine learning inference.

Set up an NPU environment

Some NPU backends require runtime dependencies or libraries. When using compiled model API, LiteRT organizes these requirements through an `Environment` object. Use the following code to find the appropriate NPU libraries or drivers:

```
// Provide a dispatch library directory (following is a hypothetical path) for the NPU
std::vector<Environment::Option> environment_options = {
    {
        Environment::OptionTag::DispatchLibraryDir,
        "/usr/lib64/npu_dispatch/"
    }
};
```

```
LITER_ASSIGN_OR_RETURN(auto env, Environment::Create( absl::MakeConstSpan(environment_options) ));
```

Runtime integration

The following code snippet shows a basic implementation of the entire process in C++:

```

// 1. Load the model that has NPU-compatible ops
LITERT_ASSIGN_OR_RETURN(auto model, Model::Load("mymodel_npu.tflite"));

// 2. Create a compiled model with NPU acceleration
// See following section on how to set up NPU environment
LITERT_ASSIGN_OR_RETURN(auto compiled_model,
    CompiledModel::Create(env, model, kLiteRtHwAcceleratorNpu));

// 3. Allocate I/O buffers
LITERT_ASSIGN_OR_RETURN(auto input_buffers, compiled_model.CreateInputBuffers());
LITERT_ASSIGN_OR_RETURN(auto output_buffers, compiled_model.CreateOutputBuffers());

// 4. Fill model inputs (CPU array -> NPU buffers)
float input_data[] = { /* your input data */ };
input_buffers[0].Write<float>(absl::MakeConstSpan(input_data, /*size*/));

// 5. Run inference
compiled_model.Run(input_buffers, output_buffers);

// 6. Access model output
std::vector<float> data(output_data_size);
output_buffers[0].Read<float>(absl::MakeSpan(data));

```

Zero-copy with NPU acceleration

Using zero-copy enables an NPU to access data directly in its own memory without the need for the CPU to explicitly copy that data. By not copying data to and from CPU memory, zero-copy can significantly reduce end-to-end latency.

The following code is an example implementation of Zero-Copy NPU with `AHardwareBuffer`, passing data directly to the NPU. This implementation avoids expensive round-trips to CPU memory, significantly reducing inference overhead.

```

// Suppose you have AHardwareBuffer* ahw_buffer

LITERT_ASSIGN_OR_RETURN(auto tensor_type, model.GetInputTensorType("input_tensor"));

LITERT_ASSIGN_OR_RETURN(auto npu_input_buffer, TensorBuffer::CreateFromAahwb(
    env,
    tensor_type,
    ahw_buffer,
    /* offset = */ 0

```

```
));

std::vector<TensorBuffer> input_buffers{npv_input_buffer};

LITERT_ASSIGN_OR_RETURN(auto output_buffers, compiled_model.CreateOutputBuffers());

// Execute the model
compiled_model.Run(input_buffers, output_buffers);

// Retrieve the output (possibly also an AHWB or other specialized buffer)
auto ahwb_output = output_buffers[0].GetAhwb();
```

Chain multiple NPU inferences

For complex pipelines, you can chain multiple NPU inferences. Since each step uses an accelerator-friendly buffer, your pipeline stays mostly in NPU-managed memory:

```
// compiled_model1 outputs into an AHWB
compiled_model1.Run(input_buffers, intermediate_buffers);

// compiled_model2 consumes that same AHWB
compiled_model2.Run(intermediate_buffers, final_outputs);
```

NPU on-device compilation caching

LiteRT supports NPU on-device (known as JIT) compilation of `.tflite` models. JIT compilation can be especially useful in situations where compiling the model ahead of time is not feasible.

JIT compilation however can come with some latency and memory overhead to translate the user-provided model into NPU bytecode instructions on-demand. To minimize the performance impact NPU compilation artifacts can be cached.

When caching is enabled LiteRT will only trigger the re-compilation of the model when required, e.g.:

- The vendor's NPU compiler plugin version changed;
- The Android build fingerprint changed;
- The user-provided model changed;
- The compilation options changed.

In order to enable NPU compilation caching, specify the `CompilerCacheDir` environment tag in the environment options. The value must be set to an existing writable path of the application.

```
const std::array environment_options = {
    litert::Environment::Option{
        /*.tag=*/litert::Environment::OptionTag::CompilerPluginLibraryDir,
        /*.value=*/kCompilerPluginLibSearchPath,
    },
    litert::Environment::Option{
        litert::Environment::OptionTag::DispatchLibraryDir,
        kDispatchLibraryDir,
    },
    // 'kCompilerCacheDir' will be used to store NPU-compiled model
    // artifacts.
    litert::Environment::Option{
        litert::Environment::OptionTag::CompilerCacheDir,
        kCompilerCacheDir,
    },
};

// Create an environment.
LITERT_ASSERT_OK_AND_ASSIGN(
    auto environment, litert::Environment::Create(environment_options));

// Load a model.
auto model_path = litert::testing::GetTestFilePath(kModelFileName);
LITERT_ASSERT_OK_AND_ASSIGN(auto model,
    litert::Model::CreateFromFile(model_path));

// Create a compiled model, which only triggers NPU compilation if
// required.
LITERT_ASSERT_OK_AND_ASSIGN(
    auto compiled_model, litert::CompiledModel::Create(
        environment, model, kLiteRtHwAcceleratorNpu));
```

Example latency and memory savings:

The time and memory required for NPU compilation can vary based on several factors, like the underlying NPU chip, the complexity of the input model etc.

The following table compares the runtime initialization time and memory consumption when NPU compilation is required versus when compilation can be skipped due to caching. On one sample device we obtain the following:

TFLite model	model init with NPU compilation	model init with cached compilation	init memory footprint with NPU compilation	init memory with cached compilation
torchvision_resnet152.tflite	7465.22 ms	198.34 ms	1525.24 MB	355.07 MB
torchvision_lraspp_mobilenet_v3_large.tflite	1592.54 ms	166.47 ms	254.90 MB	33.78 MB

On another device we obtain the following:

TFLite model	model init with NPU compilation	model init with cached compilation	init memory footprint with NPU compilation	init memory with cached compilation
torchvision_resnet152.tflite	2766.44 ms	379.86 ms	653.54 MB	501.21 MB
torchvision_lraspp_mobilenet_v3_large.tflite	784.14 ms	231.76 ms	113.14 MB	67.49 MB

Except as otherwise noted, the content of this page is licensed under the [Creative Commons Attribution 4.0 License](https://creativecommons.org/licenses/by/4.0/) (<https://creativecommons.org/licenses/by/4.0/>), and code samples are licensed under the [Apache 2.0 License](https://www.apache.org/licenses/LICENSE-2.0) (<https://www.apache.org/licenses/LICENSE-2.0>). For details, see the [Google Developers Site Policies](https://developers.google.com/site-policies) (<https://developers.google.com/site-policies>). Java is a registered trademark of Oracle and/or its affiliates.

Last updated 2026-06-16 UTC.